

## **INTRODUCTION TO BISON**

### **Objectives:**

- To understand bison tool.
- To implement the parser using bison

### **Prerequisites:**

- Knowledge of the C programming language.
- Knowledge of basic level of context free and EBNF grammars.

## **I. INTRODUCTION**

Parsing is the process of matching grammar symbols to elements in the input data, according to the rules of the grammar. The parser obtains a sequence of tokens from the lexical analyzer, and recognizes its structure in the form of a parse tree. The parse tree expresses the hierarchical structure of the input data, and is a mapping of grammar symbols to data elements. Tree nodes represent symbols of the grammar (non-terminals or terminals), and tree edges represent derivation steps.

There are two basic parsing approaches: top-down and bottom-up. Intuitively, a top-down parser begins with the start symbol. By looking at the input string, it traces a leftmost derivation of the string. By the time it is done, a parse tree is generated top-down. While a bottom-up parser generates the parse tree bottom-up. Given the string to be parsed and the set of productions, it traces a rightmost derivation in reverse by starting with the input string and working backwards to the start symbol.

Bison is a tool for building programs that handle structured input. The parser's job is to figure out the relationship among the input tokens. A common way to display such relationships is a parse tree.

Bison is a general-purpose parser generator that converts a grammar description (Bison Grammar Files) for an LALR(1) context-free grammar into a C program to parse that grammar. The Bison parser is a bottom-up parser. It tries, by shifts and reductions, to reduce the entire input down to a single grouping whose symbol is the grammar's start-symbol.

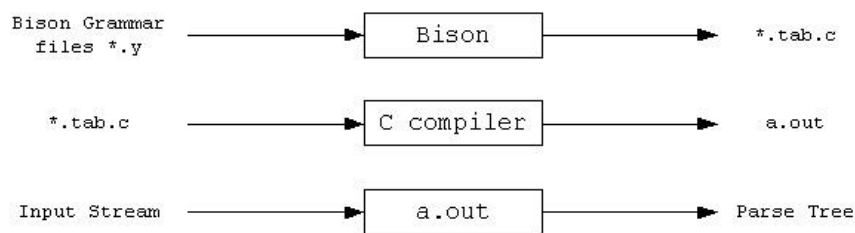


Fig. 10.1 Working of Bison

## Matching an Input using Bison Parser

A grammar is a series of rules that the parser uses to recognize syntactically valid input.

Statement:      NAME '=' expression

Expression:     NUMBER '+' NUMBER  
                  | NUMBER '-' NUMBER

The vertical bar, |, means there are two possibilities for the same symbol; that is, an expression can be either an addition or a subtraction. The symbol to the left of the: is known as the left-hand side of the rule, often abbreviated LHS, and the symbols to the right are the right-hand side, usually abbreviated RHS. Several rules may have the same left-hand side; the vertical bar is just shorthand for this. Symbols that actually appear in the input and are returned by the lexer are terminal symbols or tokens, while those that appear on the left-hand side of each rule are nonterminal symbols or non-terminals. Terminal and nonterminal symbols must be different; it is an error to write a rule with a token on the left side.

A bison specification has the same three-part structure as a flex specification. (Flex copied its structure from the earlier lex, which copied its structure from yacc, the predecessor of bison.) The first section, the definition section, handles control information for the parser and generally sets up the execution environment in which the parser will operate. The second section contains the rules for the parser, and the third section is C code copied verbatim into the generated C program.

... definition section ...

%%

... rules section ...

%%

... user subroutines section ...

The declarations here include C code to be copied to the beginning of the generated C parser, again enclosed in `%{` and `%}`. Following that are `%token` token declarations, telling bison the names of the symbols in the parser that are tokens. By convention, tokens have uppercase names, although bison doesn't require it. Any symbols not declared as tokens have to appear on the left side of at least one rule in the program. The second section contains the rules in simplified BNF. Bison uses a single colon rather than `::=`, and since line boundaries are not significant, a semicolon marks the end of a rule. Again, like flex, the C action code goes in braces at the end of each rule.

Bison creates the C program by plugging pieces into a standard skeleton file. The rules are compiled into arrays that represent the state machine that matches the input tokens. The actions have the `$N` and `@N` values translated into C and then are put into a switch statement within `yyparse()` that runs the appropriate action each time there's a reduction.

## Abstract Syntax Tree

One of the most powerful data structures used in compilers is an abstract syntax tree. An AST is basically a parse tree that omits the nodes for the uninteresting rules. A bison parser doesn't automatically create this tree as a data structure. Every grammar includes a start symbol, the one that has to be at the root of the parse tree.

## II. SOLVED EXERCISE:

Write a Bison program to check the syntax of a simple expression involving operators `+`, `-`, `*` and `/`.

```
%{
    #include<stdio.h>
    #include<stdlib.h>
}%

%token NUMBER ID NL
%left '+'
%left '*'

%%
stmt : exp NL { printf("Valid Expression"); exit(0); }
;
```

```

exp : exp '+' term
    | term
term: term '*' factor
    | factor
factor: ID
      | NUMBER
      ;
%%

```

```

int yyerror(char *msg)
{
    printf("Invalid Expression\n");
    exit(0);
}

void main ()
{
    printf("Enter the expression\n");
    yyparse();
}

```

### Flex Part

```

%{
    #include "y.tab.h"      //filename.tab.h : here both flex and bison
}%

%%
[0-9]+ {return NUMBER; }
\n {return NL ;}
[a-zA-Z][a-zA-Z0-9_]* {return ID; }
. {return yytext[0]; }
%%

```

### Steps to execute:

- Type Flex program and save it using .l extension.
- Type the bison program and save it using .y extension.
- Compile the bison code using  
**\$ bison -d filename.y**

The option **-d** Generates the file y.tab.h with the #define statements that associate the yacc user-assigned "token codes" with the user-declared "token names." This association allows source files other than y.tab.c to access the token codes.

- d. This command generates two files  
filename.tab.h and filename.tab.c
- e. Compile the flex code using  
**\$ flex filename.l**
- f. Compile the generated C file using  
**\$ gcc lex.yy.c filename.tab.c -o output**
- g. This gives an executable output.out
- h. Run the executable using **\$ ./output**

### III. LAB EXERCISES:

Write a bison program,

- 1. To check a valid declaration statement.
- 2. To check a valid decision making statements.
- 3. To evaluate an arithmetic expression involving operations +,-,\* and /.
- 4. To validate a simple calculator using postfix notation. The grammar rules are as follows –

input  $\rightarrow$  input line |  $\epsilon$

line  $\rightarrow$  '\n' | exp '\n'

exp  $\rightarrow$  num | exp exp '+'  
                  | exp exp '-'  
                  | exp exp '\*'  
                  | exp exp '/'  
                  | exp exp '^'  
                  | exp 'n'

### IV. ADDITIONAL EXERCISES:

- 1. Write a grammar to recognize strings 'aabb' and 'ab' ( $a^n b^n$ ,  $n \geq 0$ ). Write a Bison program to validate the strings for the derived grammar.

2. Write a grammar to recognize  $(a^n b, n \geq 10)$ . Write a Bison program to validate the strings for the derived grammar.